

Zakładany Plan Pracy

Temat pracy inżynierskiej

Nous – Desktopowa platforma do przeprowadzania testów psychometrycznych z synchronizacją wyników do chmury

1. Cel i zakres projektu

Celem projektu jest zaprojektowanie i implementacja wieloplatformowej aplikacji desktopowej służącej do zarządzania i uruchamiania testów psychometrycznych. Aplikacja dostarcza środowisko, w którym niezależnie opracowane testy można instalować, uruchamiać, a ich wyniki — zbierać, przechowywać lokalnie i synchronizować z bazą danych w chmurze.

Projekt zakłada realizację następujących celów:

- Stworzenie wyizolowanego, bezpiecznego środowiska uruchomieniowego dla testów (zarówno webowych, jak i natywnych Python/PsychoPy).
- Umożliwienie badaczom zarządzania zestawem testów z poziomu jednego interfejsu graficznego.
- Zaimplementowanie szyfrowania i integralności wyników badań.
- Zapewnienie synchronizacji wyników z bazą danych Firebase (Firestore).
- Opublikowanie aplikacji jako natywnego pakietu instalacyjnego na systemy Windows, macOS i Linux oraz wersji przeglądarkowej.

2. Stos technologiczny

Warstwa	Technologia
Aplikacja desktopowa	Electron v40 (Node.js + Chromium)
Język frontendu	JavaScript (ES Modules)
Baza danych chmurowa	Firebase Firestore
Uwierzytelnianie	Firebase Authentication
Lokalna baza danych	IndexedDB (przez bibliotekę <code>idb</code>)
Szyfrowanie	AES-GCM (Web Crypto API) + HMAC-SHA256 (<code>crypto</code> Node.js)
Zarządzanie kluczami	Electron <code>safeStorage</code>
Tryb HPM	Python / PsychoPy (osadzony interpreter)
Testowanie jednostkowe	Vitest + happy-dom + fake-indexeddb
Testowanie E2E	Playwright
CI/CD	GitHub Actions

Warstwa	Technologia
Strona projektu	Jekyll (GitHub Pages)
Pakowanie aplikacji	electron-builder (NSIS dla Windows, DMG dla macOS, AppImage dla Linux)

3. Planowane etapy pracy

Etap 1 — Analiza wymagań i projektowanie architektury

Na pierwszym etapie przeprowadzona zostanie analiza wymagań funkcjonalnych i нефункциональных systemu. Zdefiniowane zostaną przypadki użycia:

- badacz pobiera test z biblioteki chmurowej i go uruchamia,
- uczestnik wypełnia ankietę demograficzną przed badaniem,
- wyniki badania są zapisywane lokalnie, podpisywane i opcjonalnie synchronizowane z Firestore.

Podjęta zostanie decyzja o wyborze Electrona jako frameworka, uzasadniona koniecznością dostępu do systemu plików (zapis wyników, instalacja testów), możliwością integracji z interpreterem Python oraz wsparciem dla wielu platform. Zaprojektowana zostanie modułowa architektura aplikacji z wyraźnym podziałem na procesy: główny (`main.js`) oraz renderer (`src/`).

Zdefiniowane zostaną wymagania bezpieczeństwa: szyfrowanie wyników, walidacja źródeł pobieranych paczek ZIP, ochrona przed atakami ZIP Slip.

Etap 2 — Implementacja jądra aplikacji (proces główny)

Zaimplementowany zostanie plik `main.js` odpowiedzialny za zarządzanie cyklem życia aplikacji Electron oraz komunikację IPC między procesem głównym a rendererem.

Planowane funkcjonalności:

- **Tworzenie okna głównego:** Konfiguracja `BrowserWindow` z włączoną izolacją kontekstu i wyłączoną integracją Node.js w rendererze (bezpieczeństwo).
- **Pobieranie testów:** Obsługa kanału IPC `download-and-run`, implementacja kolejki pobierania (`downloadQueue`), obsługa przekierowań HTTP, rozpakowanie ZIP z walidacją ścieżek (ochrona przed ZIP Slip), zapis `meta.json`.
- **Uruchamianie testów JS:** Otwieranie testu w osobnym oknie pełnoekranowym (`openTestWindow`) z dedykowanym `preload_test.js`.
- **Uruchamianie testów Python (HPM):** Spawning procesu Pythona z osadzonego środowiska (`runPythonTestIfPossible`), obsługa wyników przez IPC.
- **Zarządzanie kluczem szyfrowania:** Generowanie i przechowywanie 256-bitowego klucza AES przy użyciu `safeStorage` Electrona; klucz nie jest nigdy przechowywany w postaci jawnej.
- **Podpisywanie HMAC:** Każdy zapisany wynik badania zostaje podpisany kluczem HMAC-SHA256 (wygenerowanym z klucza głównego), co umożliwia weryfikację integralności danych.
- **Lokalny zapis wyników:** Wyniki zapisywane są do pliku `.json` w folderze danych użytkownika (`app.getPath('userData')`).
- **Skanowanie biblioteki lokalnej:** Kanał IPC `get-local-versions` skanuje folder `tests_library`, odczytuje wersje z `meta.json` i informuje renderer o stanie instalacji.

- **Aktualizacja aplikacji:** Integracja `electron-updater` pobierającego nowe wersje launchera z GitHub Releases.

Zostanie zapewniona kompatybilność ze wszystkimi obsługiwanymi platformami (Windows, macOS, Linux), w tym obsługa różnych ścieżek danych na każdym z systemów.

Etap 3 — Implementacja interfejsu użytkownika i modułów frontendu

Cała logika warstwy prezentacji zostanie podzielona na moduły ES importowane przez główny punkt wejścia `src/app.js`.

Planowane moduły:

`src/modules/ui.js`

Centralne miejsce przechowywania referencji do elementów DOM (`elements`) oraz funkcji przełączania widoków (`switchView`). Umożliwia leniwe ładowanie widoków bez przeładowania strony.

`src/modules/auth.js`

Obsługa logowania, rejestracji i wylogowania przez Firebase Authentication. Wdrożenie trybu gościa (bez konta), który umożliwia korzystanie z aplikacji bez rejestracji.

`src/modules/library.js`

Główny widok biblioteki testów. Pobieranie listy testów z kolekcji Firestore `tests`, cache'owanie metadanych w `localStorage`, wyświetlanie testów w czterech trybach widoku (siatka, lista, tabela, kompaktowy). Obsługa wyszukiwania z debouncingiem, tagowania testów przez `tags.js`, aktualizacji postępu pobierania w czasie rzeczywistym.

`src/modules/database.js`

Warstwa dostępu do lokalnej bazy IndexedDB (przez bibliotekę `idb`). Przechowywanie wyników badań, szablonów ankiet i historii synchronizacji.

`src/modules/cryptoService.js`

Serwis szyfrowania oparty o Web Crypto API. Implementacja szyfrowania AES-GCM danych wynikowych przed zapisem do bazy chmurowej. Klucz dostarczany przez proces główny przez bezpieczny kanał IPC.

`src/modules/results.js`

Odbiór wyników badania od okna testowego (IPC `test-results-forwarded`), inicjacja wyświetlenia ankiety demograficznej, zapis do IndexedDB, opcjonalny eksport CSV i synchronizacja z Firestore.

`src/modules/demographics.js`

Wyświetlanie ankiet demograficznych opartych na szablonach. Zbieranie danych uczestnika przed każdym badaniem.

`src/modules/demoCreator.js`

Kreator szablonów ankiet demograficznych z graficznym interfejsem. Obsługa typów pól: tekst, liczba, lista rozwijana, checkboxy, radio, data. Import i export szablonów do pliku JSON.

`src/modules/history.js`

Widok historii wyników. Przeglądanie, filtrowanie i eksport zebranych wyników do pliku CSV.

`src/modules/sync.js`

Usługa synchronizacji wyników do Firestore. Obsługa trybu offline — dane są kolejkowane lokalnie i synchronizowane po odzyskaniu połączenia. Toggle automatycznej synchronizacji z persistencją w `localStorage`.

`src/modules/settings.js`

Ustawienia aplikacji: motyw (dark/light/user), kolor akcentu, rozmiar czcionki. Zapisywane w `localStorage` i aplikowane przy starcie.

`src/modules/appUpdater.js`

Frontend do obsługi aktualizacji launchera. Sprawdzanie dostępności nowej wersji, wyświetlanie powiadomienia, pobieranie i instalacja aktualizacji.

`src/modules/whatsNew.js`

Wyświetlanie historii zmian i nowości w aplikacji. Po przez wyświetlanie plików readme.md z repozytorium projektu na GitHub.

`src/modules/dialog.js`

Globalny system okienek dialogowych (alert, confirm) niezależny od natywnych `window.alert` — umożliwia spójny wygląd na wszystkich platformach.

`src/modules/utils.js`

Funkcje pomocnicze: `debounce`, `escapeHtml`, `sortByInstallStatus`, `getLocalVersionsCached`, `flattenObject` (do spłaszczania hierarchicznych wyników na potrzeby CSV).

`src/modules/tags.js`

System tagowania testów. Przechowywanie tagów w `localStorage`, menu zarządzania tagami, parser zapytań wyszukiwania z obsługą tagów.

Etap 4 — Bezpieczeństwo danych wynikowych

Realizacja wymagań bezpieczeństwa jest kluczowym elementem systemu, gdyż aplikacja operuje na danych z badań naukowych.

Planowane mechanizmy:

- **Szyfrowanie wyników** — dane wynikowe przed zapisem do Firestore szyfrowane są algorytmem AES-GCM kluczem 256-bitowym zarządzanym przez `safeStorage`. Każdy rekord posiada unikalne IV (Initialization Vector).
 - **Integralność plików lokalnych** — każdy wynik zapisywany na dysk jest podpisany HMAC-SHA256. Umożliwia to weryfikację, czy plik nie był modyfikowany od chwili zapisu.
 - **Walidacja pobieranych paczek** — przed rozpakowaniem paczki ZIP sprawdzane są ścieżki wszystkich wpisów w celu wykrycia ataków ZIP Slip. URL musi spełniać jednocześnie dwa warunki: (1) protokół HTTPS, (2) hostname to `github.com` lub `raw.githubusercontent.com` i ścieżka zaczyna się od `/KaucBartosz/`. Jedynym wyjątkiem jest `objects.githubusercontent.com` (CDN GitHub Releases), który używa haszowanych URL niezawierających nazwy właściciela. Dzięki temu uniemożliwione jest wskazanie jako źródło zasobów spoza repozytorium autora.
 - **Walidacja ID testów** — identyfikatory testów walidowane są wyrażeniem regularnym `[a-zA-Z0-9_-]+` przed użyciem jako ścieżka systemu plików.
 - **Izolacja kontekstu** — okna testów uruchamiane są z wyłączoną integracją Node.js i włączoną izolacją kontekstu; komunikacja wyłącznie przez preload API.
-

Etap 5 — Tryb wysokiej precyzji (HPM)

Wiele testów psychometrycznych wymaga dokładności pomiaru czasu rzędu pojedynczych milisekund, której web renderer nie jest w stanie zapewnić. Aby temu zaradzić, zdefiniowany zostanie Tryb Wysokiej Precyzji (HPM — High Precision Mode). Pozwoli on również na uruchamianie testów napisanych w języku Python, co pozwoli na korzystanie z zewnętrznej aparatury badawczej.

Planowany sposób realizacji:

- Pobieranie i instalacja osadzonego środowiska Python (ok. 300 MB) jako jednorazowej operacji.
 - Wykrycie obecności pliku `main.py` w folderze testu.
 - Uruchamianie testu jako procesu potomnego Pythona (`spawn`) zamiast okna przeglądarki.
 - Obsługa wyników przez lokalny socket IPC lub stdout.
 - Obsługa specyfiki poszczególnych dystrybucji Linux (detekcja Debian/Ubuntu vs. Fedora/RHEL i informowanie użytkownika o wymaganych zależnościach systemowych).
 - Sprawdzanie i aktualizacja silnika HPM przez Firestore.
-

Etap 6 — Wersja przeglądarkowa

Aplikacja zostanie przygotowana do uruchomienia bezpośrednio w przeglądarce internetowej (bez instalacji) z zachowaniem możliwie dużej funkcjonalności.

Planowane ograniczenia wersji przeglądarkowej:

- Brak dostępu do systemu plików — wyniki eksportowane natychmiast jako plik CSV.
- Brak widoku historii i aktualizacji (nie dotyczy wersji web).
- Brak synchronizacji i opcji wylogowania.
- Brak trybu HPM.
- Automatyczne logowanie jako gość bez ekranu logowania.

- Blokada urządzeń dotykowych (tablety/smartfony) — platforma wymaga klawiatury i myszy.

Etap 7 — Strona informacyjna projektu

Stworzona zostanie strona informacyjna projektu hostowana na GitHub Pages, zbudowana z użyciem generatora stron statycznych Jekyll.

Planowana zawartość:

- Strona główna z opisem projektu i przyciskiem pobierania najnowszej wersji.
- Strona funkcjonalności z opisem możliwości platformy.
- Automatyczne osadzanie numeru najnowszej wersji i linków do plików instalacyjnych przez GitHub API.

Etap 8 — Testowanie

Weryfikacja poprawności implementacji zostanie przeprowadzona na dwóch poziomach.

Testy jednostkowe (Vitest)

Testy obejmą następujące moduły:

Plik testowy	Testowany moduł
<code>utils.test.js</code>	<code>debounce</code> , <code>escapeHtml</code> , <code>flattenObject</code> , <code>sortByInstallStatus</code>
<code>cryptoService.test.js</code>	Szyfrowanie i deszyfrowanie AES-GCM
<code>database.test.js</code>	CRUD na IndexedDB (fake-indexeddb)
<code>dialog.test.js</code>	Tworzenie i zamykanie okienek dialogowych
<code>library.test.js</code>	Renderowanie kart testów, filtrowanie, sortowanie
<code>results.test.js</code>	Obsługa wyników i zapis do bazy
<code>settings.test.js</code>	Zastosowanie ustawień motywu i koloru
<code>ui.test.js</code>	Przełączanie widoków
<code>whatsNew.test.js</code>	Ładowanie widoku nowości

Środowisko testowe: Vitest z `happy-dom` (emulacja DOM) i `fake-indexeddb` (emulacja IndexedDB).
Raportowanie pokrycia kodu przez `@vitest/coverage-v8`.

Testy End-to-End (Playwright)

Testy E2E uruchomią rzeczywistą aplikację Electron i będą symulować działania użytkownika: logowanie, nawigację, pobieranie testu, wyświetlanie wyników.

Etap 9 — CI/CD i publikacja

Wdrożony zostanie potok GitHub Actions realizujący następujące zadania:

- **release.yml** — Automatyczne budowanie instalatorów (.exe dla Windows, .AppImage dla Linux, .dmg dla macOS) po opublikowaniu nowego Release na GitHub i wgranie ich jako artefaktów wydania, co umożliwi automatyczne aktualizacje przez **electron-updater**.
- **update-web-tests.yml** — Automatyczne pobieranie aktualnych wersji testów z ich repozytoriów i commit do gałęzi, zapewnienie aktualności wersji przeglądarkowej.
- **hpm-packs.yml** — Budowanie i publikowanie paczek silnika HPM.

Konfiguracja **electron-builder** zapewni:

- Tworzenie instalatora NSIS (Windows) z możliwością wyboru katalogu instalacji.
- Tworzenie pakietu DMG (macOS).
- Tworzenie paczki AppImage (Linux).
- Różnicowe pakiety aktualizacji (**differentialPackage**).

4. Struktura katalogów projektu

```
Nous/
├─ main.js           # Proces główny Electron (IPC, pobieranie, HPM,
szyfrowanie)
├─ preload.js        # API wystawione rendererowi okna głównego
├─ preload_test.js   # API wystawione oknu testowemu
├─ index.html        # Główny plik HTML aplikacji
├─ style.css         # Globalny arkusz stylów
├─ src/
│   └─ app.js         # Punkt wejścia frontendu, inicjalizacja modułów
│   └─ firebaseConfig.js # Konfiguracja Firebase SDK
│   └─ modules/       # Moduły biznesowe
│       ├── appUpdater.js
│       ├── auth.js
│       ├── cryptoService.js
│       ├── database.js
│       ├── demoCreator.js
│       ├── demographics.js
│       ├── dialog.js
│       ├── history.js
│       ├── library.js
│       ├── recaptcha.js
│       ├── results.js
│       ├── settings.js
│       ├── sync.js
│       ├── tags.js
│       ├── ui.js
│       ├── updates.js
│       ├── utils.js
│       └─ whatsNew.js
└─ tests/
    ├── unit/         # Testy jednostkowe Vitest
    └─ e2e/           # Testy E2E Playwright
```

```
|— docs/                # Strona GitHub Pages (Jekyll)
|— .github/workflows/   # Potoki CI/CD
|— package.json         # Konfiguracja projektu i electron-builder
```

5. Harmonogram realizacji

Etap	Zakres prac
1	Analiza wymagań, wybór technologii, projekt architektury
2	Implementacja procesu głównego Electron (IPC, pobieranie, bezpieczeństwo)
3	Implementacja modułów frontendu (UI, biblioteka, baza danych, wyniki)
4	Mechanizmy bezpieczeństwa danych (szyfrowanie AES-GCM, HMAC, walidacja ZIP)
5	Tryb HPM (integracja Python/PsychoPy)
6	Wersja przeglądarkowa (adaptacja ograniczeń środowiska web)
7	Strona informacyjna projektu (Jekyll / GitHub Pages)
8	Testowanie (Vitest — testy jednostkowe, Playwright — testy E2E)
9	CI/CD, pakowanie i publikacja wydań (GitHub Actions, electron-builder)

6. Spodziewane wyniki

Po realizacji wszystkich etapów system będzie spełniał następujące wymagania:

- Aplikacja instaluje się i działa na systemach Windows, macOS i Linux.
- Badacz może przeglądać i zarządzać biblioteką testów synchronizowaną z chmurą.
- Każdy test uruchamiany jest w izolowanym, pełnoekranowym środowisku.
- Wyniki badania są szyfrowane i podpisane kryptograficznie, zapewniając poufność i integralność danych.
- Wersja przeglądarkowa umożliwia przeprowadzenie badania bez instalacji aplikacji.
- Automatyczny system aktualizacji informuje użytkownika o nowych wersjach launchera i testów.
- Pokrycie kodu testami jednostkowymi jest mierzalne i raportowane przy każdym push.